Multi-Agent Fact-Checking Debate System

Step-by-Step Build Plan: Concept Used and Why, Per Step

Build Plan for Hottam

July 2026

Contents

Multi-Agent Fact-Checking Debate System — Build Plan

A Proponent agent and a Skeptic agent research and argue opposite sides of a claim in parallel, over multiple rounds; a Judge scores each round and decides whether to conclude or continue. Every debate is a persisted, streamable, traced LangGraph run.

Step 1 — Set up the project environment

Create the project skeleton (`agent/`, `api/`, `data/`, `tests/`), a virtualenv, and install `langgraph`, `langchain`, `langchain-openai`, `faiss-cpu`, `fastapi`, `uvicorn`, `langgraph-checkpoint-sqlite`, `langgraph-checkpoint-postgres`, `langsmith`. Add a `.env` with your OpenAI key and LangSmith keys.

Concept used & why: No LangGraph concept yet — this is plain project hygiene. It matters because every later step assumes this structure exists; skipping it means restructuring mid-build later.

Step 2 — Design the state schema

Define `DebateState` (claim, round, max_rounds, proponent_args, skeptic_args, verdicts, final_verdict, citations, messages) as a `TypedDict`, with list fields marked `Annotated[list[X], operator.add]`.

Concept used & why: State + Reducers (Video 4 core concepts, Video 6 parallel reducers). The state is the single shared contract every node reads/writes — designing it first means every node you write afterward has a fixed, known interface. `operator.add` reducers are used specifically because multiple nodes (Proponent, Skeptic, and repeated Judge rounds) will each append to these lists in the same run; without a reducer, later writes would silently overwrite earlier ones.

Step 3 — Build the graph skeleton with stub nodes

Wire `START → round_manager → (proponent, skeptic) → judge → conditional → round_manager | finalize → END` using dummy functions that return fake data, and compile it.

Concept used & why: StateGraph, nodes, edges (Video 4). Building the skeleton with stubs first — before any real LLM logic — lets you prove the control-flow topology (the loop terminates, the parallel branches both run, the fan-in works) at zero API cost, instead of debugging topology and prompts at the same time.

Step 4 — Add the parallel fan-out for the two debaters

Connect `round_manager` to both `proponent` and `skeptic` with `add_edge`, and connect both back into a single `judge` node.

Concept used & why: Parallel workflows / fan-out-fan-in (Video 6). The Proponent and Skeptic don't depend on each other's output within the same round — they research and argue independently — so running them in the same super-step is both faster and mirrors how a real debate works (each side prepares before hearing the final rebuttal). The Judge node acts as the implicit join, only running once both branches complete.

Step 5 — Add the conditional loop that controls debate rounds

Write `route_debate(state)`, a routing function returning `"finalize"` if the Judge marked the debate `concluded` or `max_rounds` is reached, else `"round_manager"`. Wire it with `add_conditional_edges`.

Concept used & why: Conditional edges + cycles (Videos 7 and 8). This is the single feature that makes LangGraph necessary here instead of a plain LangChain chain — a debate is inherently a loop with a data-dependent exit condition. The hard `max_rounds` check alongside the LLM's own `concluded` flag is deliberate: never trust the LLM alone to end a cycle, since a stubborn or confused Judge could loop forever otherwise.

Step 6 — Build the RAG tool over a seed evidence corpus

Load a small set of documents, chunk with `RecursiveCharacterTextSplitter`, embed and index in `FAISS`, and wrap retrieval in an `@tool`-decorated `evidence_search` function with a clear docstring.

Concept used & why: RAG-as-a-tool (Video 19). Exposing retrieval as a tool rather than a fixed pre-fetch step means each debater's LLM decides for itself when it actually needs evidence, and can issue multiple refined searches within one argument — more realistic than forcing one retrieval pass per turn.

Step 7 — Add a live web-search tool via MCP

Stand up or connect to an MCP server exposing a search tool, and connect with `MultiServerMCPClient`; add its discovered tools alongside `evidence_search`.

Concept used & why: MCP tool integration (Video 18). Claims often need current information your static corpus doesn't have. Sourcing this tool via MCP instead of a hand-written `@tool` function decouples "who built the tool" from "who uses it" — the same pattern used to connect any external tool server without custom integration code.

Step 8 — Build the Proponent and Skeptic subgraphs

Each is its own compiled `StateGraph(DebaterState)` with a `chat_node ⇄ tools` loop (`ToolNode` + `tools_condition`) that ends in a structured-output `Argument`. Test each standalone with `.invoke()` before touching the parent graph.

Concept used & why: Subgraphs (Video 21) **+ the ReAct tool-calling loop** (Video 17). Subgraphs let you build and unit-test each debater as an independent, reusable unit with its own clean state, rather than cramming both personas into one tangled node. The tool-calling loop inside each subgraph is what lets the LLM decide, turn by turn, whether to search for more evidence or commit to its argument.

Step 9 — Build the Judge node with structured output

Write a node that takes both arguments plus prior rounds and calls `llm.with_structured_output(RoundVerdi` producing a typed `winner`, `reasoning`, and `concluded` flag.

Concept used & why: Structured output (Video 6/7 pattern, `with_structured_output` + Pydantic). A judge that returns free text is unreliable to route on programmatically; forcing a typed schema means `route_debate` in Step 5 can safely read `verdict.concluded` as a real boolean instead of parsing prose.

Step 10 — Replace stubs with real nodes and stress-test the loop

Swap Phase 3's stub functions for the real `proponent_node`, `skeptic_node`, and `judge` from Steps 8 to 9. Run several test claims end-to-end and read full transcripts.

Concept used & why: This is where **Steps 4, 5, 8, and 9 combine** into the actual working agent loop. Testing here (before persistence/streaming/deployment) isolates bugs to agent logic rather than infrastructure — the same reasoning behind building the stub skeleton first in Step 3.

Step 11 — Add persistence with a checkpointer

Compile the graph with `InMemorySaver` first, then switch to `SqliteSaver` once behavior is stable, scoping every debate to its own `thread_id`.

Concept used & why: Checkpointing / persistence (Video 10, extended in Video 14). Each debate becomes resumable and independently inspectable (`get_state_history`), and a crash mid-debate no longer means re-running from scratch — the checkpointer already has every completed round saved.

Step 12 — Add streaming

Use `workflow.stream(..., stream_mode="updates")` to emit progress events per node ("Proponent researching…", "Judge deliberating…") and optionally `stream_mode="messages"` for token-level output.

Concept used & why: Streaming (Video 12). Without it, a caller stares at a blank screen for the 10 to 30+ seconds a multi-round debate takes; streaming node-level updates gives real-time visibility into which stage is currently running.

Step 13 — Add a human moderator checkpoint

In the Judge node, if a round comes back `"inconclusive"` after 2+ rounds, call `interrupt({...})` to ask a human for a tiebreaking fact or ruling, and resume later with `Command(resume={...})`.

Concept used & why: Human-in-the-loop (Video 20). This gives the system a safety valve for genuinely deadlocked debates instead of forcing an unreliable automatic verdict, and demonstrates that `interrupt()` can gate a *decision*, not just a *tool call*.

Step 14 — Add observability and an evaluation set

Confirm `LANGCHAIN_TRACING_V2=true` produces full nested traces for each debate, then build a 15 to 20-claim dataset with known verdicts and a LangSmith evaluator to score `final_verdict` accuracy after every prompt change.

Concept used & why: LangSmith tracing and evaluation (Videos 15 and 16). A graph this branchy (parallel + conditional + cyclic + tool calls) is very hard to debug from print statements alone; tracing shows exactly which node/tool call caused a bad outcome, and the evaluation set turns prompt tuning from guesswork into measurement.

Step 15 — Wrap the graph in a FastAPI backend

Expose `POST /debates` (start a debate, return `thread_id`), `GET /debates/{id}/stream` (SSE stream of progress), and `GET /debates/{id}` (current state/verdict), all calling the same compiled `workflow`.

Concept used & why: Separation of agent logic from delivery (the same principle behind Video 11's Streamlit wrapper). The graph itself stays UI-agnostic — any client just needs to call `.invoke()`/`.stream()` with the right `thread_id`.

Step 16 — Containerize and deploy

Write a `Dockerfile` for the API, a `docker-compose.yml` adding a Postgres service, swap `SqliteSaver` for `PostgresSaver` pointing at it, and deploy the container (with Postgres) to a host of your choice with secrets set as environment variables, not baked into the image.

Concept used & why: This is the production-hardening of **Step 11's persistence pattern** — same checkpointer interface, durable backend. Containerizing ensures the environment that worked in development is exactly what runs in production, removing "works on my machine" failure modes.

When you’re ready, we start writing real code at Step 1.